

Shortest Paths: the Bellman Ford Algorithm

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Single-Source Shortest Paths: Algorithms

- Easy cases that we can solve in linear time:
 - Unweighted graphs: use **breadth-first search**.
 - Directed acyclic graphs (DAGs): use **dynamic programming**.
- If edge costs are nonnegative, we'll use **Dijkstra's algorithm**.
- If some edge costs are negative, we use the **Bellman-Ford algorithm**.
- If there is a negative cycle, we are out of luck!

The Bellman-Ford Algorithm

```
c[s] = 0, c[j ≠ s] = Infinity
```

```
Repeat n-1 times:
```

```
    Tighten every edge in the graph
```

```
    If  $c[i] + \text{cost}(i, j) < c[j]$  for any edge  $(i, j)$ 
```

```
        Output "negative cycle detected!"
```

- $O(mn)$ running time.
- Detects negative cycles.
- Easy Analysis: after k iterations of the main loop, each label $c[j]$ reflects the cost of the shortest k -hop path from s to j .
(Can also interpret as a dynamic programming algorithm)